WEB101 & WEB102 Practical Assignments

Introduction

You will work in groups to progressively develop a functional web application that covers the learning outcomes for both modules.

The assignments are designed to build upon each other, with each new assignment extending the functionality implemented in previous ones. By the end of the course, you will have developed a complete web application with both frontend and backend components.

Project Reviews and Presentations

- **First Progress Review (April 18, 2026)**
  - Brief presentation of project status and achievements so far
  - Discuss challenges encountered
  - Receive feedback
  - Identify any areas requiring additional guidance or resources

- **Second Progress Review (May 9, 2026)**
  - Demonstrate key functionality completed since first review
  - Review remaining tasks and timeline to completion
  - Address any challenges
  - Receive guidance for final implementation phase

- **Final Submission and Presentation (May 21, 2026)**
  - Final project demonstration to panel members
  - Code review session
  - Showcase complete functional solution
  - Q&A session on all aspects of implementation

WEB101: Web Application Fundamentals

Assignment 2: Frontend Foundation & Application Structure

Introduction

In this first assignment, you will establish the foundation for your web application by setting up the development environment, implementing core routing functionality, and creating essential UI components. This assignment focuses on building a strong foundation/structure that will support the additional features in upcoming assignments.

Learning Outcomes

LO1: Describe fundamental concepts of web development (package managers, runtime environments, frameworks)

LO3: Develop a multiple-page application using routing techniques

LO7: Design and implement reusable components adhering to single responsibility principle

Project Overview

1. **Project Planning:**
    - Select an application to develop (either a new concept or based on an existing app)
    - Define the core functionalities that will showcase the learning outcomes
    - Discuss a basic project plan outlining the features to be implemented across all assignments
2. **Development Environment Setup:**
    - Set up a React/Next.js frontend framework
    - Set up a remote repository on GitHub for code collaboration between your team members
3. **Application Structure & Routing:**
    - Document the file-based routing structure in the README file
    - Implement navigation to reusable components using slugs. (example. For instagram a reusable component for the reels page could ingest the data and be reused multiple times based on the data given from the server) https://nextjs.org/docs/14/app/building-your-application/routing/dynamic-routes

    - Set up route guards/protection for pages that will require authentication later

4. **Component Development:**
    - Plan and document on the readme file on the creation of the reusable UI components based on your topic

5. **Documentation:**
    - Document the project setup and architecture in the README.md
    - Include clear instructions for setting up the development environment
    - Document each component with its purpose, props/inputs, and usage examples
    - Add comprehensive code comments explaining complex logic
        - Penalty: per commit and branching strategy not being adhered would result in a mark deducted per offense

Submission

GitHub Repository containing your working web application including a descriptive README.md. Please note that as the assignment doesn't require a report submission, ensure that the comments successfully suffice in supporting your thought process through this assignment. You are required to define the functionality in your README.md that your application would achieve.

Date of Completion:

1st April, 2026

Note: Please note that as the assignment doesn't require a report submission, ensure that the comments successfully suffice in supporting your thought process through this assignment. You are required to define the functionality in your README.md that your application would achieve.

Grading Criteria

- Code Organization & Readability (2 marks)
- Code Comments & Documentation (1 mark)
- Functional Requirements (5 marks)
- User Interface & User Experience (2 mark)

Assignment 3: State Management & API Integration

Introduction

Building on the foundation established in Assignment 2, this assignment focuses on implementing state management solutions and integrating with APIs. You will enhance your application with data fetching capabilities, manage application state, and handle user interactions with data.

Learning Outcomes

LO2: Explain differences between communication patterns (request/response, publish/subscribe)

LO4: Handle web requests to interact with third-party APIs, including file uploads and pagination

LO6: Develop and implement strategies for managing application-level and component-level state

Project Overview

1. **State Management Implementation:**
    - Implement Zustand as your application's state management solution
    - Create a minimum of three separate stores/contexts for distinct domains (e.g., authentication, product data, user preferences)
    - Develop clear unidirectional data flow patterns with actions, reducers, and selectors
2. **API Service Layer:**
    - Build a structured service layer that abstracts API communication from components:
        - Implement the repository pattern with separate repositories for each resource type
        - Create at least five custom React hooks (e.g., useProducts, useAuth) for common API operations
    - Configure Axios or Fetch with interceptors that:
        - Handle authentication tokens
        - Standardize error responses
        - Process request/response transformations
3. **Data Fetching & Display:**

- Integrate with your backend APIs using Tanstack Query (React Query) for:

    - Automatic refetching
    - Optimistic updates

- Implement specific UI states for each data fetch operation:
    - Loading skeletons or spinners
    - Error messages with retry options
    - Empty state indicators

- Utilize Next.js Suspense boundaries with fallback UI components for improved loading experiences

4. Form Handling & Validation:

- Implement form solutions using Tanstack Form

- Create validation rules with clear, user-friendly error messages for all form fields

- Develop file upload functionality that includes:
    - Drag-and-drop interface
    - File type and size validation
    - Upload progress indicators
    - Preview capabilities for images and documents

- Design form submission processes with appropriate success/failure feedback and loading states

5. Pagination & Filtering:

- Implement server-side pagination with configurable page size

- Create a data table or list view with:
    - Multi-field sorting (ascending/descending)
    - Field-specific filtering options (dropdown, date range, text search)

- Implement either infinite scrolling with intersection observers or a "load more" button with clear loading indicators

Date of Completion: 26th April 2026

Note: Please note that as the assignment doesn't require a report submission, ensure that the comments successfully suffice in supporting your thought process through this assignment. You are required to define the functionality in your README.md that your application would achieve.

Grading Criteria

- Code Organization & Readability (2 marks)
- Code Comments & Documentation (1 mark)
- Functional Requirements (5 marks)
- User Interface & User Experience (2 mark)

Submission Penalty

- A penalty of 1 mark each day will be applied for late submission.

Assignment 4: Authentication & Real-time Features

Introduction

In this assignment, you will implement authentication mechanisms, protected routes, and real-time features in your application. You will also integrate data visualization capabilities to

present time-series data in an engaging way.

Learning Outcomes

LO5: Employ token-based and session-based authentication methods

LO10: Develop web applications that integrate real-time data communication (websockets, server-sent events)

LO11: Animate data visualizations that present time series data using charting libraries

Project Overview

1. **Authentication Implementation:**

- Build a complete authentication system with:
    - Registration form with email verification and password strength requirements
    - Login form with "remember me" functionality and failed attempt handling
    - Password reset workflow with email verification and secure link expiration

- Implement JWT-based authentication with HTTP-only cookies or secure localStorage

- Create token refresh mechanisms that handle expiration without disrupting user experience

- Develop route protection using Next.js middleware or React Router guards that:
    - Redirect unauthenticated users to login
    - Handle authorization based on user roles
    - Preserve intended destination after successful login

1. **User Management:**

- Construct a user profile section that allows editing of:
    - Personal information (name, email, profile picture)
    - Account security settings (password change, two-factor authentication)
    - Communication preferences

- Implement role-based access control with at least three distinct roles (e.g., Admin, Editor, User)

- Create a user settings page with customizable options for:
    - Theme preferences (light/dark mode)
    - Notification settings

    - Display preferences

- Develop session management features including:
    - Clear logout functionality that removes tokens/cookies
    - Automatic logout after 30 minutes of inactivity
    - Session expiration notifications with renewal options

2. Real-time Communication:

- Integrate native WebSockets with proper connection handling

- Build one of the following real-time features:
    - Chat system with message history, read receipts, and typing indicators
    - Notification center showing user-specific alerts in real-time
    - Live dashboard with auto-updating metrics and status indicators
    - Collaborative editing with concurrent user presence indicators

3. Error Handling & Edge Cases:

- Implement React Error Boundaries at strategic component levels to:
    - Prevent entire application crashes
    - Display user-friendly fallback UIs

- Design specific error states with:
    - Clear explanations of what went wrong
    - Actionable recovery options (retry, refresh, contact support)
    - Contextual help related to the error

- Address these specific edge cases:
    - Network disconnection during form submission
    - Empty data states with appropriate messaging
    - Browser compatibility issues with graceful degradation

Date of Completion:

18th May, 2026

Note: Please note that as the assignment doesn't require a report submission, ensure that the comments successfully suffice in supporting your thought process through this assignment. You are required to define the functionality in your README.md that your application would achieve.

Grading Criteria

- Code Organization & Readability (2 marks)
- Code Comments & Documentation (1 mark)
- Functional Requirements (5 marks)
- User Interface & User Experience (2 mark)

Submission Penalty

- A penalty of 1 mark each day will be applied for late submission.

Final Project: Deployment, Optimization & Finalization

Introduction

The final project brings together all components developed in the previous assignments and focuses on deployment, optimization, and finalization of your web application. You will set up continuous deployment, and ensure your application is production-ready.

Learning Outcomes

LO8: Compose a basic continuous deployment pipeline

LO9: Configure a domain name and link it to a web service

Integration of all previous learning outcomes (LO1-LO7, LO10-LO11)

Project Overview

A. Project Proposal

Create a comprehensive project proposal document containing:

1. **Aim and Objective**
    - Clear statement of the project's purpose and goals
    - Alignment with learning outcomes
    - Specific, measurable objectives

2. **Feasibility Analysis**
    - Technical feasibility assessment
    - Resource requirements and constraints
    - Risk assessment and mitigation strategies

3. **Expected Outcomes**
    - Detailed description of deliverables
    - Success criteria for the project
    - Impact and significance of the project

4. **Work Plan**
    - Timeline with milestones
    - Task distribution among team members
    - Resource allocation

B. Code Assessment & Review

1. **Application Finalization:**
    - Complete all core functionalities from previous assignments
    - Fix any outstanding bugs or issues
    - Implement feedback from previous assignments
    - Ensure all features work together cohesively

2. **Managed Deployment and Hosting:**
    - Set up your application on Render cloud platform
    - Configure automatic deployments from your GitHub repository

3. **Performance Optimization:**
    - Ensure responsive design across devices

4. **Final Documentation:**
    - Complete user documentation
    - Commenting for the codebase
    - Document deployment procedures

Date of Completion:

22nd May, 2026

Note: Please note that as the assignment doesn't require a report submission, ensure that the comments successfully suffice in supporting your thought process through this assignment. You are required to define the functionality in your README.md that your application would achieve.

Grading Criteria

- Project Proposal (3 marks):
    - Aim and Objective (1)
    - Expected Outcome (1)
    - Work Plan (1)

- Code Assessment & Review (37 marks):
    - Code Organization & Readability (2)
    - Code Comments & Documentation (2)
    - Functional Requirements (15)
    - User Interface & User Experience (15)
    - Deployment & Configuration (3)

Penalties

[-50% of overall grade of Project] - Student is unable to justify and explain code logic of his/her implementation

Submission Penalty

- A penalty of 1 mark each day will be applied for late submission.

WEB102: Server Application Fundamentals

Assignment 1: API Design & Server Setup

Introduction

In this first backend assignment, you will design and implement the fundamental API architecture for your web application. You will create a server-side application, define RESTful endpoints, and establish the foundation for your backend services.

Learning Outcomes

LO1: Explain differences between request-response and event-driven API design paradigms

LO3: Design RESTful APIs using appropriate URIs, HTTP methods, response codes, and metadata

LO7: Develop a server-side application using frameworks and create API endpoints

1. API Design & Planning

- Design a comprehensive API schema that defines at least 5 core resources for your application

- Create detailed API contract documentation that includes:
  - Complete endpoint definitions with URL paths, parameters, and query strings
  - Request and response examples in JSON format for each endpoint
  - Authentication requirements for protected endpoints

- Define all resource models with:
  - Required and optional fields with their data types
  - Validation rules for each field
  - Clear relationship mappings (one-to-many, many-to-many)

- Establish a specific API versioning strategy:
  - Either URL-based
  - Header-based

2. Server Setup & Configuration

- Set up an Express.js or NestJS server with TypeScript support

- Configure a development environment with:
  - Nodemon for automatic server restart
  - Environment variables management (.env files)
  - ESLint and Prettier for code quality

- Establish a project structure that includes separate directories for:
  - Controllers (handling HTTP requests)
  - Services (containing business logic)
  - Models (defining data structures)
  - Routes (defining API endpoints)
  - Middleware (request processing)

- Implement a middleware pipeline that includes:
  - Request logging (timestamp, method, path)
  - Body parsing for JSON and form data

    - CORS configuration with specific allowed origins
    - Central error handling middleware

3. Basic Endpoint Implementation

- Create fully functional RESTful endpoints for the core resources.

- Implement all appropriate HTTP methods with request body validation

- Configure response status codes.

- Develop a consistent error response format that includes:
    - Error code
    - Human-readable message
    - Details about validation failures when applicable

4. Request Validation

- Implement data validation for all endpoints using:
    - Joi, Yup, or class-validator library
    - Custom validators for complex business rules

- Create detailed validation schemas for:
    - Request bodies (for POST and PUT requests)
    - URL parameters (for resource IDs)
    - Query parameters (for filtering and pagination)

- Configure specific error messages for validation failures that:
    - Identify the exact field with the error
    - Explain the validation rule that failed
    - Provide guidance on correct input format

5. Code Structure & Documentation

- Organize code following separation of concerns with:
    - Controllers that handle HTTP requests only
    - Services that contain all business logic
    - Data access layer for database operations
    - Utility functions in separate files

- Document all major design decisions in a README.md file:
    - API architecture choices
    - Authentication approach

    - Error handling strategy
    - Database schema design

- Add JSDoc comments for all functions and classes that include:
    - Description of functionality
    - Parameter descriptions with types
    - Return value descriptions
    - Usage examples where appropriate

- Create initial API documentation using:
    - Example requests and responses for each endpoint
    - Authentication instructions
    - Error code explanations

Submission

GitHub Repository containing your working web application including a descriptive README.md. Please note that as the assignment doesn't require a report submission, ensure that the comments successfully suffice in supporting your thought process through this assignment. You are required to define the functionality in your README.md that your application would achieve.

Date of Completion:

17th April, 2026

Grading Criteria

- Code Organization & Readability (2 marks)
- Code Comments & Documentation (1 mark)
- Functional Requirements (6 marks)
- Deployment & Configuration (1 mark)

Submission Penalty

- A penalty of 1 mark each day will be applied for late submission.

Assignment 2: Database Integration & Advanced Endpoints

Introduction

Building on the API foundation from Assignment 1, this assignment focuses on integrating database systems, implementing advanced endpoint functionality, and applying best practices for API scalability.

Learning Outcomes:
LO2: Implement HTTP protocol, handle file uploads, pagination, and searching

LO4: Apply best practices for designing scalable APIs

LO9: Integrate with databases using relevant tools like ORMs

Project Overview

Database Integration:

- Set up a PostgreSQL database with:
  - Proper schema design including tables, relationships, and constraints
  - Migration scripts for version control of database changes
  - Separate development and testing environments

- Implement Prisma ORM for database interaction:
  - Define comprehensive Prisma schema models for all entities
  - Configure relations (one-to-many, many-to-many) with proper cascading options
  - Set up proper indexing for frequently queried fields
  - Implement database seeding for development and testing environments

Advanced Endpoint Functionality:

- Create file upload endpoints that:
  - Handle multipart form data for file submission
  - Validate file types, sizes, and quantities
  - Store files securely with proper naming conventions
  - Provide file metadata and URLs in API responses

- Implement paginated endpoints that:
  - Support configurable page size
  - Include total count and pagination metadata
  - Allow cursor-based pagination for large datasets

- Develop search and filter capabilities:
  - Text search across specified fields
  - Category and status filtering

- Add sorting functionality that:
  - Supports multiple sort fields (e.g., `?sort=createdAt:desc,name:asc`)
  - Provides default sorting options
  - Ensures consistent ordering for pagination

Query Optimization:

- Optimize Prisma queries using:
  - Selective field inclusion to minimize data transfer

    - Explicit transaction handling for multi-operation requests

- Implement data fetching strategies:
    - Batch operations for related data updates
    - Chunked queries for large data operations

- Create database indices for:
    - Foreign keys and join columns
    - Frequently filtered fields
    - Text search columns

Testing & Validation:

- Create a comprehensive Postman collection that:
    - Includes sample requests for API endpoints
    - Provides properly formatted request bodies for each endpoint
    - Documents expected responses for successful and error cases
    - Organizes endpoints into logical folders by functionality

- Document the complete happy flow for user acceptance testing:
    - Detail step-by-step user journeys for core functionality
    - Include expected outcomes at each step
    - Provide screenshots of successful operations
    - Document acceptance criteria for each user story

Date of Completion:

9th May, 2026

Note: Please note that as the assignment doesn't require a report submission, ensure that the comments successfully suffice in supporting your thought process through this assignment. You are required to define the functionality in your README.md that your application would achieve.

Grading Criteria

- Code Organization & Readability (2 marks)
- Code Comments & Documentation (1 mark)
- Functional Requirements (6 marks)
- Deployment & Configuration (1 mark)

Submission Penalty

- A penalty of 1 mark each day will be applied for late submission.

Assignment 3: Authentication, Security & Real-time Communication

Introduction

In this third assignment, you will implement authentication systems, security measures, and real-time communication capabilities in your backend application. You will secure your API endpoints and establish the foundation for dynamic, real-time features.

Learning Outcomes

LO6: Identify and mitigate common security vulnerabilities in API architectures

LO8: Implement authentication and authorization mechanisms for protected endpoints

LO10: Implement real-time data communication using WebSockets

Project Overview

1. **Authentication System:**

- Build a complete user registration and login system with:
    - User registration endpoint with basic validation
    - Login functionality with secure credential validation

- Apply industry-standard password security:
    - Hash passwords using bcrypt
    - Enforce minimum password complexity requirements

- Develop middleware for authorization that:
    - Verifies JWT signature and validity
    - Checks user roles against required permissions
    - Returns appropriate 401/403 responses when access is denied

- Implement data access filters that:
    - Restrict users to only their own data
    - Filter query results based on user permissions

2. **Security Measures:**

- Protect against these specific vulnerabilities:
    - SQL/NoSQL injection via parameterized queries
    - Broken authentication via account lockouts

- Implement comprehensive request validation:
    - Schema validation for all request payloads
    - Data type checking and sanitization
    - Protection against oversized payloads

3. **Real-time Communication:**

- Configure a WebSocket server using Socket.io or ws:
    - Implement secure WebSocket connections (wss://)
    - Create authentication for socket connections
    - Handle connection lifecycle (connect, disconnect, reconnect)

- Build a publish/subscribe system with:
    - Topic-based channels for broad updates
    - User-specific channels for personal notifications
    - Room-based distribution for group communications

- Develop these event handlers:
    - Data change notifications (create/update/delete)

    - System broadcasts for maintenance or announcements

- Create standardized message formats for:
    - Data synchronization payloads
    - Notification messages
    - Error messages

4. **Error Handling & Logging:**

- Implement a structured error handling system:
    - Domain-specific error classes with error codes
    - Consistent error response format across all endpoints
    - Detailed internal errors with sanitized client responses

- Configure request/response logging that captures:
    - Request method, URL, and headers (excluding sensitive data)
    - Response status code and timing information

- Develop an error reporting mechanism that:
    - Alerts on critical or frequent errors (loggings)
    - Provides context for debugging (stack traces, request info)
    - Respects privacy by excluding sensitive data

Date of Completion:

19rd May, 2026

Note: Please note that as the assignment doesn't require a report submission, ensure that the comments successfully suffice in supporting your thought process through this assignment. You are required to define the functionality in your README.md that your application would achieve.

Grading Criteria

- Code Organization & Readability (2 marks)
- Code Comments & Documentation (1 mark)
- Functional Requirements (6 marks)
- Deployment & Configuration (1 mark)

Submission Penalty

- A penalty of 1 mark each day will be applied for late submission.

Final Project: Documentation & Deployment

Introduction

The final project brings together all components developed in the previous assignments and focuses on API gateway configuration, comprehensive documentation, and deployment of your backend application.

Learning Outcomes

LO5: Compose developer resources (API documentation, samples, tools)

LO11: Construct and configure key components of an API platform, including API gateways

Integration of all previous learning outcomes (LO1-LO4, LO6-LO10)

Project Overview

A. Project Proposal

1. **Aim and Objective**

- Write a clear problem statement explaining what specific API functionality your project addresses

- Define 3-5 concrete objectives that are measurable and time-bound (e.g., "Implement a gateway that handles 500 requests/second")

2. **Feasibility Analysis**

- Evaluate your team's technical capabilities against project requirements

- Identify and document specific resource needs:
    - Server/cloud requirements with estimated costs
    - Third-party services or APIs with usage limits

    - Development tools and environments

- Create a prioritized risk register with mitigation strategies for your top 3 technical challenges

3. **Expected Outcomes**

- Detail the final deliverables with acceptance criteria for each:
    - Documentation deliverables with completion standards
    - Deployment artifacts and environments

- Define success indicators that will demonstrate your API platform's effectiveness

B. API Platform Enhancement

Since your core API is already developed, focus on enhancing it with this production-level improvements:

1. **Production Deployment**

- Finalize your multi-environment deployment pipeline

- Implement secure environment variable and secrets management

- Complete any remaining API endpoints needed for a full-featured product

- Add production-grade security measures (HTTPS enforcement, CSP headers, etc.)

C. Developer Resources

1. **Documentation**

- Create a logically structured API reference that follows industry standards

- Provide comprehensive database documentation:
    - Database ERD reflecting the final database architecture
    - Complete schema design with column types,constraints, and indices

- Provide clear examples for all endpoints with request/response pairs

2. **Developer Tools**

- Integrate your API with RAML and generate RAML specification that:
    - Defines all API endpoints, request parameters, and response schemas
    - Includes detailed data types and examples
    - Documents security schemes and authentication requirements
    - Maintains synchronization between implementation and documentation

Date of Completion:

21st May, 2026

Note: Please note that as the assignment doesn't require a report submission, ensure that the comments successfully suffice in supporting your thought process through this assignment. You are required to define the functionality in your README.md that your application would achieve.

Grading Criteria

- Project Proposal (3 marks):
  - Aim and Objective (1)
  - Expected Outcome (1)
  - Work Plan (1)

- Code Assessment & Review (22 marks):
  - Code Organization & Readability (2)
  - Code Comments & Documentation (2)
  - Functional Requirements (15)
  - Deployment & Configuration (3)

- API Documentation (10 marks):
  - Clarity & Coherence (2)
  - Documentation (4)
  - API Tooling (2)
  - Deployment & Configuration (2)

Penalties

[-50% of overall grade of assignment] - Student is unable to justify and explain code logic of his/her implementation

Submission Penalty

- A penalty of 1 mark each day will be applied for late submission.

Practical Assessment I, II, III, IV Rubric:

Criteria	Description	Weightage	Accomplished(A)	Good(B)	Satisfactory(C)	Needs Improvement(D)	Unsatisfactory(E)
Code Organization & Readability	Code structure, formatting, naming conventions, and overall organization principles	2.0%	Exceptional code organization with clear structure. Consistently follows best practices for formatting and naming. Code is highly maintainable and modular.	Well-organized code following most conventions. Good structure and modularity. Easy to follow.	Adequate code organization with basic structure. Some inconsistencies in formatting or naming. Generally readable.	Poor code organization with unclear structure. Inconsistent formatting and naming. Hard to follow.	Chaotic code organization. No clear structure or conventions followed. Unreadable.
Code Comments & Documentatio n	Quality and usefulness of documentation, comments, and supporting materials	1.0%	Comprehensive, clear documentation with relevant comments explaining complex logic. Complete setup and usage instructions included.	Good documentation with helpful comments. Basic setup and usage instructions provided.	Basic documentation and comments present. Minimal but sufficient instructions included.	Limited documentation or unhelpful comments. Unclear instructions.	Missing or inadequate documentation. No useful comments or instructions.
Functional Requirements	Completion and correctness of required features and functionality	5.0%	All requirements implemented correctly with additional enhancements. Robust error handling and edge	All core requirements implemented correctly. Basic error handling present.	Most requirements implemented. Some minor issues or missing features. Basic	Several requirements are incomplete or incorrectly implemented. Major functionality	Most requirements are not met. Core functionality broken or missing.

			cases covered.		functionality works.	issues.	
User Interface & User Experience	Design implementation , responsiveness , and overall user experience	1.0%	Polished, intuitive interface with excellent responsiveness and user flow. Attention to detail in design implementation.	Good interface design with consistent styling and responsivenes s. Clear user flows.	Functional interface with basic styling and responsiveness. Standard user flows implemented.	Poor interface with inconsistent styling or responsiveness issues. Confusing user flows.	Broken interface with major styling or responsiveness problems. Poor user experience.
Deployment & Configuration	Setup, configuration, and deployment of the application	1.0%	Perfect deployment and configuration. All features working in a production environment. Clear deployment documentation.	Successful deployment with working features. Basic configuration documented.	Basic deployment complete with core features working. Minimal configuration documentation.	Deployment issues affecting functionality. Incomplete configuration.	Failed deployment or major configuration issues preventing usage.

Final Project Assessment IV Rubric:

A. **Project Proposal (5%)**

Criteria	Description	Weightage	Accomplished(A)	Good(B)	Satisfactory(C)	Needs Improvement(D)	Unsatisfactory(E)
Aim and Objective	Clarity and relevance of project goals and objectives	1.0%	Exceptionally clear, specific, and well-defined objectives that align perfectly with course outcomes.	Clear objectives with good alignment to course outcomes.	Basic objectives stated with adequate clarity.	Vague or poorly defined objectives.	Missing or irrelevant objectives.
Feasibility	Realistic scope and technical achievability within timeframe	1.0%	Excellent analysis of technical requirements with realistic timeline and resource planning.	Good assessment of scope with reasonable timeline.	Basic feasibility assessment with general timeline.	Limited consideration of technical constraints.	No feasibility analysis or unrealistic scope.
Expected Outcome	Clear definition of deliverables and success criteria	2.0%	Comprehensive list of measurable outcomes with detailed success criteria.	Well-defined outcomes with clear success metrics.	Basic outcomes defined with simple success criteria.	Vague outcomes with unclear success criteria.	Missing or unrealistic outcomes.
Work Plan	Project timeline, milestones, and resource allocation	1.0%	Detailed timeline with clear milestones, tasks, and resource planning.	Good timeline with defined milestones and tasks.	Basic timeline with general task breakdown.	Incomplete timeline with missing elements.	No work plan or unrealistic timeline.

B. Code Assessment & Review (35%)

Criteria	Description	Weightage	Accomplished(A)	Good(B)	Satisfactory(C)	Needs Improvement(D)	Unsatisfactory(E)
Code Organization & Readability	Structure, organization, and maintainability of codebase	2.0%	Exceptional code organization with industry-standard practices. Highly maintainable and scalable.	Well-organized code following good practices. Clear structure.	Adequate organization with basic structure. Generally readable.	Poor organization with inconsistent structure.	Chaotic organization. Unreadable code.
Code Comments & Documentation	Quality of documentation and inline comments	2.0%	Comprehensive documentation with clear, relevant comments. Complete setup guides.	Good documentation with helpful comments. Basic guides included.	Basic documentation and essential comments present.	Limited documentation or unhelpful comments.	Missing or inadequate documentation.
Functional Requirements	Implementation of proposed features and functionality	15.0%	All features implemented excellently with additional enhancements. Robust error handling.	All core features are implemented well. Good error handling.	Most features are implemented adequately. Basic functionality works.	Several features are incomplete or faulty.	Most features are missing or non-functional.
Deployment & Configuration	Production deployment and technical setup	1.0%	Perfect deployment with comprehensive configuration and documentation.	Successful deployment with good configuration.	Basic deployment with working configuration.	Deployment issues present.	Failed deployment or major issues.